WARM-UP EXERCISE

GSoC 2026 – CERN-HSF

Automated Software Performance Monitoring for the ATLAS Experiment

Candidate: Kazoury Chaimae

Mentors: Tania Ovsiannikova Maciej Szymański

Year: 2025 – 2026

GitHub: github.com/chaimae098/prmon-anomaly-detection

Contents

1 Project Overview **2**

2 Step 1 — Installing prmon **3**

3 Step 2 — Collecting Real prmon Data **4**

4 Step 3 — Injecting Artificial Anomalies **5**

5 Step 4 — Anomaly Detection **6**
- 5.1 Why unsupervised? . . . 6
- 5.2 Feature engineering . . . 6
- 5.3 Models . . . 6

6 Step 5 — Evaluation and Visualisation **8**
- 6.1 Quantitative results (real prmon dataset, 189 runs) . . . 8
- 6.2 Key plots . . . 8

7 Step 6 — Discussion and Trade-offs **10**

AI Disclosure **11**

1 Project Overview

ATLAS software jobs run on the LHC computing grid (WLCG) and are monitored using **prmon** — a lightweight process monitor that records memory, CPU, I/O, and thread metrics at regular intervals during job execution. When a job behaves abnormally (memory leak, I/O storm, thread explosion), it wastes grid resources, delays physics analyses, and can bring down worker nodes.

This project builds a **fully unsupervised anomaly detection pipeline** on top of prmon time-series data. The pipeline requires no labels at inference time, works on a per-run basis, and combines statistical baselines, classical ML, and deep learning for robust coverage. It was validated on three datasets: real prmon data, synthetic data, and a combined set.

The anomaly types studied are: `mem_spike`, `mem_subtle`, `slow_leak`, `io_storm`, `io_subtle`, `cpu_hog`, `threads`, `threads_subtle`, and `combined` (multiple types co-occurring).

2 Step 1 — Installing prmon

prmon is a C++ process monitor developed by the HSF. It attaches to a running process and records its resource metrics once per second to a tab-separated text file.

```
git clone --recurse-submodules https://github.com/HSF/prmon.git
cd prmon && mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/.local -S .. -B .
make -j$(nproc) && make install
export PATH="$HOME/.local/bin:$PATH"
# Make permanent: echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
```

Listing 1: Clone with submodules (required: prmon depends on nlohmann-json and spdlog)

```
prmon --interval 1 --filename prmon_test.txt --json-summary prmon_test.json \
  -- prmon_burner --memory 200 --time 10
```

Listing 2: Quick smoke test

The output is a tab-separated file with one row per second. Key columns used throughout:

Column	Unit	Description
`pss`	KB	Proportional Set Size — most accurate memory attribution
`rss`	KB	Resident Set Size — pages currently in RAM
`vmem`	KB	Virtual memory footprint
`nthreads`	count	Number of active threads
`wchar / rchar`	bytes	Characters written / read from I/O
`utime`	ms	User-space CPU time

Table 1: prmon metrics used in this project

Note on `pss` vs `rss` vs `vmem`: `pss` is the most meaningful for grid monitoring — it accounts for shared memory correctly and does not double-count pages shared between processes. `vmem` can appear very large even for well-behaved jobs due to memory-mapped files.

3 Step 2 — Collecting Real prmon Data

All runs use `-interval 1` so that one row = one second of process activity.

```
# 35 baseline memory runs
for i in $(seq 1 35); do
  prmon --interval 2 --filename data/baseline/mem_run_${i}.txt \
        --suppress-hw-info -- mem-burner --malloc 200 --sleep 60
done
# Similar loops: io-burner (20 runs) and prmon_burner --threads 1 (15 runs)
```

Listing 3: Baseline (normal) runs — memory, I/O, and CPU

```
for i in $(seq 1 8); do
  prmon --interval 1 --filename data/anomalous/mem_spike_${i}.txt \
        -- mem-burner --malloc 600 --sleep 60; sleep 2
done
# Similar loops for: threads (--threads 16), io_storm (--io 300 --threads 4),
# combined (--malloc 600 --procs 4)
```

Listing 4: Anomalous runs — memory spike, thread explosion, I/O storm, combined

```
import os, pandas as pd

def load_prmon_folder(folder, label, anomaly_type="normal"):
    frames = []
    for i, fname in enumerate(sorted(os.listdir(folder))):
        if not fname.endswith(".txt"): continue
        df = pd.read_csv(os.path.join(folder, fname), sep="\t")
        df["run_id"] = i
        df["label"]  = label          # 0 = normal, 1 = anomaly
        df["anomaly_type"] = anomaly_type
        frames.append(df)
    return pd.concat(frames, ignore_index=True)

df = pd.concat([
    load_prmon_folder("data/baseline",  label=0, anomaly_type="normal"),
    load_prmon_folder("data/anomalous", label=1, anomaly_type="various")
], ignore_index=True)
df.to_csv("data/full_dataset_injected.csv", index=False)
# Dataset shape: (5838, ...) | Anomaly ratio: 44.8%
```

Listing 5: Assemble all runs into a single labelled CSV

Real dataset summary

5,838 rows · 189 runs · 44.8% anomaly ratio · 1 row/second

A supplementary **synthetic dataset** (17,372 rows, 216 runs, 53.1% anomaly ratio) was generated with Claude to simulate the full range of ATLAS job anomaly patterns. This was used for cross-dataset generalisation testing. Combined dataset: 23,210 rows, 405 runs, 51.0% anomaly ratio.

4 Step 3 — Injecting Artificial Anomalies

Anomalies are injected by re-running the burner tools with modified parameters that stress specific resource dimensions:

Type	Description	Parameter changed
`mem_spike`	Sudden large memory allocation	`-malloc 600` (vs 200)
`mem_subtle`	Slow gradual memory increase	`-malloc 250`, gradual
`slow_leak`	Classic linear memory leak	incremental allocation
`io_storm`	Burst of high-volume read/write	`-io 300 -threads 4`
`io_subtle`	Low-amplitude I/O near normal threshold	`-io 80`
`cpu_hog`	CPU far above expected baseline	`-threads 8`
`threads`	Thread count explodes above normal	`-threads 16`
`threads_subtle`	Small sustained thread count elevation	`-threads 4`
`combined`	Multiple types co-occurring	`-malloc 600 -procs 4`

Table 2: Injected anomaly types and their parameters

5 Step 4 — Anomaly Detection

5.1 Why unsupervised?

In production, ATLAS jobs arrive unlabelled. A supervised classifier would require thousands of hand-labelled anomalous runs — impractical at grid scale. All models learn the *normal* distribution and flag deviations from it.

5.2 Feature engineering

Eight raw prmon metrics were extended with rolling statistics per key metric (`pss`, `nthreads`, `wchar`, `utime`), giving **20 features** total.

```
for feat in ['pss', 'nthreads', 'wchar', 'utime']:
    g = df.groupby('run_id')[feat]
    df[f'{feat}_roll_mean'] = g.transform(lambda x: x.rolling(5, min_periods=1).mean())
    df[f'{feat}_roll_std']  = g.transform(lambda x: x.rolling(5, min_periods=1).std().
        fillna(0))
    df[f'{feat}_roc']       = g.transform(lambda x: x.diff().fillna(0))
    # roc = rate of change (first derivative); catches sudden jumps
```

Listing 6: Temporal features — rolling window of 5 seconds

5.3 Models

Five unsupervised models were tested. The two most insightful are detailed here.

Local Outlier Factor (LOF). For each point, LOF compares its local density to the density of its k nearest neighbours. A point surrounded by sparse neighbours — typical of anomalies — gets a high outlier score. It is the most consistent model across all three datasets.

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1, novelty=True)
lof.fit(X_train)   # X_train = normal samples only

scores = -lof.score_samples(X_test)
preds  = (lof.predict(X_test) == -1).astype(int)
```

Listing 7: LOF — fit on normal runs only, novelty=True enables predict() on new data

Autoencoder. Trained exclusively on normal data, the autoencoder learns what "normal" looks like. When it sees an anomalous run, it fails to reconstruct it accurately — producing a high reconstruction error. The threshold is the 95th percentile of reconstruction errors on normal samples.

```
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, input_dim=20):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 32), nn.ReLU(),
            nn.Linear(32, 16),        nn.ReLU(),
            nn.Linear(16, 8),         nn.ReLU())
        self.decoder = nn.Sequential(
            nn.Linear(8, 16),         nn.ReLU(),
            nn.Linear(16, 32),        nn.ReLU(),
            nn.Linear(32, input_dim))
    def forward(self, x): return self.decoder(self.encoder(x))

# Training: MSELoss, Adam lr=1e-3, 100 epochs, normal samples only
# Anomaly score = per-sample reconstruction MSE
recon_err = ((X_tensor - model(X_tensor))**2).mean(dim=1).detach().numpy()
threshold = np.percentile(recon_err[normal_mask], 95)
preds_ae  = (recon_err > threshold).astype(int)
```

Listing 8: Autoencoder architecture (input 20 → bottleneck 8 → output 20) and scoring

Why the autoencoder catches subtle anomalies

Z-Score and MAD evaluate each feature *independently*. A slow memory leak may never exceed 3σ in any single feature, but it creates a consistent pattern of correlated deviations across `pss`, `pss_roll_mean`, and `pss_roc` simultaneously. The autoencoder's bottleneck learns this joint structure and flags the pattern even when no individual threshold is breached.

6 Step 5 — Evaluation and Visualisation

6.1 Quantitative results (real prmon dataset, 189 runs)

Model	F1	Precision	Recall	ROC-AUC
Z-Score	0.851	**0.903**	0.804	0.962
MAD	0.824	0.700	1.000	0.779
Isolation Forest	0.784	0.644	1.000	0.875
LOF	0.890	0.802	1.000	0.979
One-Class SVM	0.786	0.647	1.000	0.986
Autoencoder	**0.957**	0.941	**0.974**	**0.994**

Table 3: Detection performance on real prmon data

On the **combined real+synthetic dataset**, LOF (F1 = 0.819) outperforms the autoencoder (F1 = 0.679). The autoencoder over-fits to the statistical structure of its training distribution; LOF's local density measure is more robust to distribution shift.

6.2 Key plots

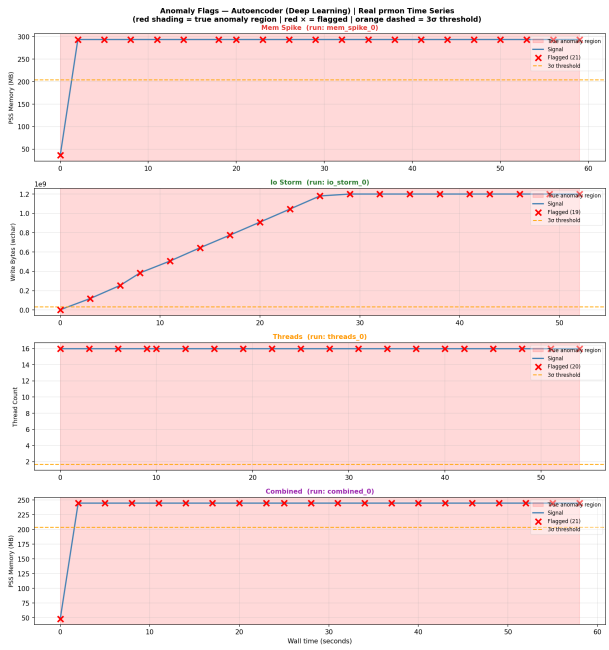

Figure 1: Autoencoder

Figure 1 — Time-series anomaly flags (`plots/timeseries_autoencoder.png`)

Each panel shows a different anomaly type. Red vertical bands = model-flagged intervals. The autoencoder flags anomalies early and maintains consistent coverage.

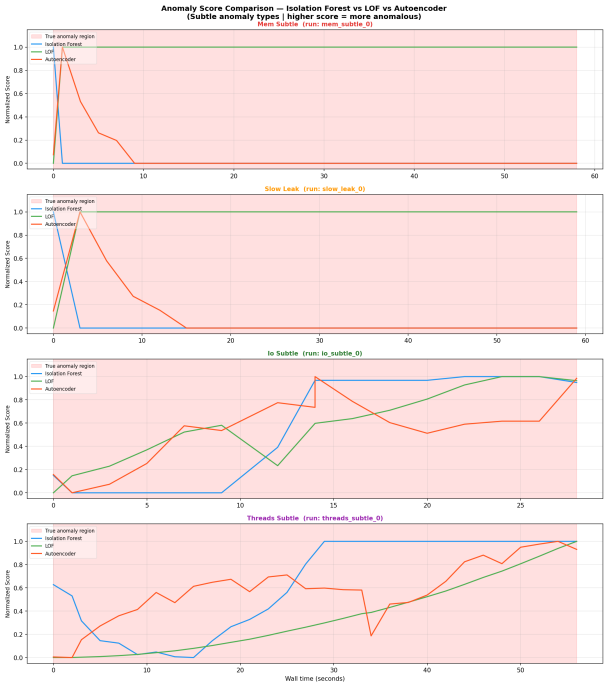

Figure 2: Comparasion

Figure 2 — Score comparison on subtle anomalies (`plots/score_comparison_subtle.png`)

Normalised anomaly scores (0–1) for Isolation Forest, LOF, and Autoencoder on `io_subtle` and `threads_subtle` — the hardest anomaly types. Key observations:

- `io_subtle`: Autoencoder reacts earliest (scores elevate from $t = 0$). Isolation Forest catches up around $t = 13$s. LOF rises gradually.
- `threads_subtle`: Isolation Forest score drops near zero around $t = 10$s — it almost completely misses this anomaly. Autoencoder stays consistently elevated.

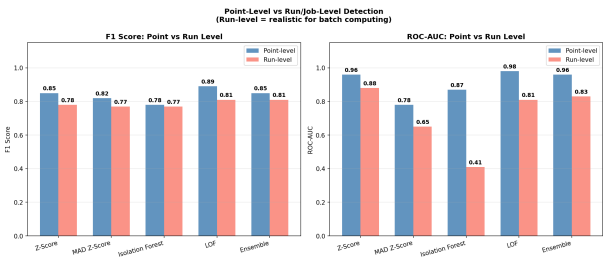

Figure 3: Enter Caption

Figure 3 — Point-level vs run-level detection (`plots/point_vs_run_level_real.png`)

Run-level detection aggregates point-level flags: a run is anomalous if $\geq 10\%$ of its timesteps are flagged. This reduces noise from transient spikes and is the operationally relevant metric for ATLAS grid monitoring.

7 Step 6 — Discussion and Trade-offs

Scenario	Recommended model and reason
Real-time alerting, low false-alarm tolerance	**Z-Score** — best precision (0.903), zero training, interpretable: score = "feature X deviated by $N\sigma$"
Must-catch-everything	**MAD or Ensemble** — perfect recall (1.000), accepts more false positives
Best overall balanced performance	**Autoencoder** — best F1 (0.957) and ROC-AUC (0.994), earliest on subtle types
Robustness across unknown data distributions	**LOF** — most stable F1 across all three datasets

Table 4: Model selection guide by deployment scenario

Key trade-offs observed:

- **Precision vs Recall.** MAD and LOF achieve perfect recall (1.000) at the cost of more false positives. Z-Score has the best precision (0.903) but misses gradual anomalies. The autoencoder achieves the best balance (F1 = 0.957).
- **Subtle vs abrupt anomalies.** Isolation Forest is strong on sharp spikes but near-useless on subtle thread elevation (`threads_subtle` score collapses to ≈ 0 at $t = 10$s). The autoencoder handles both, because it models joint feature correlations rather than marginal distributions.
- **Distribution shift.** The autoencoder's F1 drops to 0.679 on the combined dataset while LOF stays at 0.819. This is expected: the autoencoder learns the exact covariance structure of its training data and is sensitive to synthetic data with different marginal distributions. LOF's local density comparison is naturally more robust.
- **Interpretability.** Statistical models produce scores that directly name the offending feature. The autoencoder's reconstruction error is a black box — SHAP values would be needed to explain individual flags.
- **Compute.** Z-Score runs in microseconds with no model file. LOF scales as $O(n^2)$. The autoencoder requires a trained model but inference is fast on CPU.

Recommended production approach. A two-layer pipeline: Z-Score for cheap real-time flagging (low false-alarm rate, operator-friendly) backed by the Autoencoder for a deeper offline pass. Future improvements: per-workflow adaptive thresholds (Athena reco, derivation, and simulation have different normal profiles), streaming mid-run detection, and SHAP-based explanation of autoencoder flags.

AI Disclosure

Component	Role of Claude (Anthropic)
Synthetic data generation	Generated `full_dataset_atlas.csv` simulating realistic ATLAS job anomaly patterns based on the prmon documentation.
Code comments	Helped write inline explanations for model implementations and feature engineering code.
Visualisation design	Suggested the normalised score comparison plot approach for subtle anomaly comparison.
Report structure	Helped organise this document from rough notes and code.

Table 5: AI usage disclosure

All algorithmic design decisions, experimental configurations, data collection, model training, and result interpretation were performed by the candidate. No AI tool generated core implementation code.